EECS 221: Machine Learning for Science

Data–Compute Trade-offs in Neural PDE Forecasting

Suha Kagan Kose

Mid-Project Report (November 12)

Summary

The abstract set three milestones for this check-in. Their status:

1. **Data pipeline validated against The Well's released checkpoints: numerical agreement obtained.** Our data loading and metric code reproduce the paper's published test VRMSE for the released CNextU-Net and TFNO to within 1%, and recover rollout errors in the same reported range ("> 10"). The released TFNO needs the forward pass it was trained with, which differs from the corrected one our runs use (Section 1).
2. **Budgets fixed from pilot timings: done.** Batch 64, fp32, learning rate 10^{-3} for every model (from an equal-budget search), and 4-hour budgets measured in active GPU training time, with allocated GPU time reported separately.
3. **First 10%-data results: done.** All six 10% runs (3 models $\times$ 2 seeds) are trained and evaluated on the full test set at 1 and 4 hours (Section 4). After 4 hours CNextU-Net is the most accurate in developed convection and the most stable in rollouts; AViT matches it one step ahead but degrades fastest over a rollout; TFNO is too smooth. A revised plan is in Section 5.

1 Setup

Data and task. `rayleigh_benard` from The Well [1]: 1,750 trajectories of 200 frames on a 512×128 grid, 35 (Rayleigh, Prandtl) regimes, official 80/10/10 splits. From four frames of buoyancy, pressure and both velocity components, a model predicts the next frame and is rolled out autoregressively. Fields are normalised with The Well's training statistics.

Models. The Well's own implementations, trained from random initialisation, with matched parameter counts (Table 1). All three use GELU activations. Released-checkpoint reproduction and training with corrected code must be distinguished, because the same weights do not imply the same forward computation. In The Well's code, commit `2d8d7eaf` (2024-06-03, "Added gradient checkpointing") called TFNO's Fourier blocks without keeping their output, so the model computed only its lifting and projection layers; commit `e4301602` (2025-11-17, "actually use the TFNO blocks") restored the assignment [6]. The paper's TFNO results and the released TFNO checkpoint (uploaded 2025-03-28) date from that period. For the pipeline check (Section 2) we scored the released TFNO weights with the forward pass they were trained with (Fourier blocks skipped): it reproduces the published value; with the corrected forward pass the same weights score $\approx$10.5. Our own runs use The Well at commit `e63dec8`, which includes the fix, so our TFNO uses its Fourier layers (software: neuraloperator 0.3.0, PyTorch 2.8.0; our repository at `f42b2e2` for training and `4c0d3a8` for evaluation). Upstream issue #49 reports related inconsistencies between released checkpoints and published numbers [6].

Model	Family	Parameters	Normalisation	Relation to released checkpoint
CNextU-Net [2]	ConvNeXt U-Net, 4 stages	18.58M	LayerNorm	same architecture
TFNO [3]	Fourier operator, 16×16 modes	19.28M	none	same configuration; ours runs the Fourier layers (fixed forward), the released weights need the pre-fix forward
AViT [4]	axial-attention ViT	18.57M	GroupNorm, LayerNorm	none released; width 384 → 432

Table 1: The three model families.

Training. All models use AdamW, one-step MSE in normalised field values, batch size 64, and fp32. An equal-budget validation search selected a peak learning rate of 10^{-3} for each model. Training uses 500 warmup steps followed by cosine decay to zero over four hours of active GPU training. The detailed comparison with the reference recipe is in Appendix A. Checkpoints are saved at one and four hours. The one-hour checkpoint is an intermediate state of the four-hour schedule (learning rate approximately 85% of peak), not a separately optimised one-hour run. Thus, the study measures progress along a shared training schedule; it does not establish the best performance achievable with an independently tuned one-hour schedule.

Evaluation and metric conventions. VRMSE is dimensionless. For one window (one predicted frame) and one field f it is computed by The Well's metric code (`the_well.benchmark.metrics.VRMSE` at `e63dec8`) as

$$\mathrm{VRMSE}_f = \sqrt{\frac{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_{f,i} - y_{f,i})^2}{s_f^2 + \varepsilon}}, \qquad s_f^2 = \frac{1}{N-1}\sum_{i=1}^{N}(y_{f,i} - \bar{y}_f)^2, \qquad \varepsilon = 10^{-7},$$

over the $N = 512 \times 128$ grid points of the target frame. The four fields (buoyancy b, pressure p, and the velocity components u, w) are averaged with equal weights; reported values are means over windows with equal weight per window (one-step: all 196 windows of each of the 175 test trajectories; rollouts: per step, over trajectories and start frames). Two variants are reported, with the same formula and the same averaging. They differ only in the field values passed in: the *paper protocol* scores normalised fields, $(y_f - \mu_f)/\sigma_f$ with The Well's training statistics ($\sigma_f = 0.249, 0.149, 0.208, 0.180$ for b, p, u, w); our *physical-field VRMSE* scores the fields as stored. Without ε the two would be identical, since the ratio is invariant to an affine rescaling of each field. With ε, normalising multiplies s_f^2 by $1/\sigma_f^2$ (16–45), so the floor shrinks from 10^{-7} to the equivalent of $\varepsilon\sigma_f^2 \approx 2$–$6 \times 10^{-9}$. During the first ∼50 frames the velocity is nearly zero (exactly zero at $t = 0$) and its spatial variance is comparable to 10^{-7}, so the physical-field score is damped there and the normalised one is not. This is the whole difference between the released CNextU-Net's 0.669 (paper protocol) and 0.224 (physical fields): on onset windows they score 2.510 and 0.766, while from frame 50 on both give 0.038 (our 10% model: 0.0499 in both).

Failure horizon. For a rollout, the first step whose field-mean physical-field VRMSE exceeds a threshold (1, 2 or 5, all reported). Rollouts start at frames 20, 80 and 140 of every test trajectory and run 30 steps; rollouts that never cross a threshold are right-censored at step 30 and summarised by Kaplan–Meier survival curves.

2 Milestone 1: pipeline validated against the released checkpoints

We loaded The Well's released CNextU-Net and TFNO weights and scored them with our own data path and metric on every window of the 175 test trajectories (Table 2).

	Paper-protocol one-step VRMSE		Rollout from frame 0, steps 6–12 / 13–30	
Model	Ours (released weights)	Published	Ours	Published
CNextU-Net	0.669	0.670	36 / 35	> 10 / > 10
TFNO	0.663	0.657	142 / 169	> 10 / > 10

Table 2: Released checkpoints scored by our pipeline against the paper's Tables 2 and 3 (test split).

Three preliminary observations shape how we report results:

- **Aggregate relative error is sensitive to the early frames** (Fig. 1). For the first ∼50 frames the fluid is nearly at rest; the evaluated models have large relative velocity errors there, and copying the last frame (persistence) is nearly exact. From frame 50 on, the released CNextU-Net scores 0.038, against an all-window mean of 0.224 in physical fields (0.669 under the paper protocol). Near-zero target variability can amplify relative error, so this does not establish that absolute errors are largest during onset. We therefore also report onset (first input frame < 50) and developed convection separately.
- **Rollout error varies substantially with Rayleigh number.** At Ra $= 10^{10}$ the released CNextU-Net's median over steps 13–30 is 1.7; at 10^6 it is 69.
- **The fixed test evaluation is the clearer reproduction target.** The paper's validation values use 20 random batches per epoch; exact agreement would additionally require the same sampled batches and evaluation state.

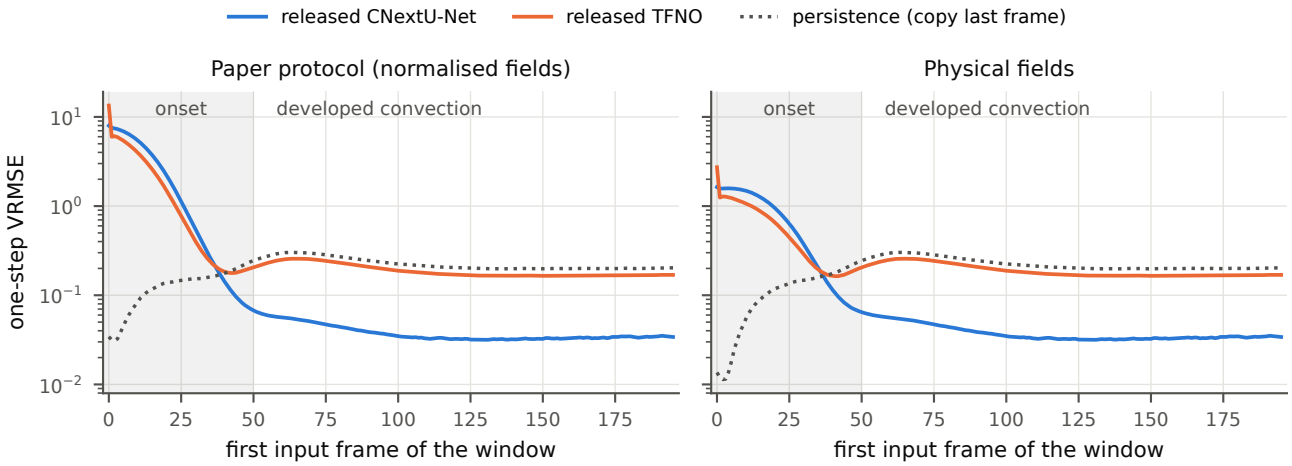


Figure 1: Released checkpoints' one-step VRMSE by window on the test set, mean over the 175 trajectories, scored under the paper protocol (left) and on physical fields (right). Shaded: convection onset (first input frame < 50). Persistence copies the last input frame.

3 Milestone 2: budgets fixed from pilot timings

Resource accounting. *Active GPU training time* is the measured training time counted against the experimental budget. *Allocated GPU time* is the duration for which a job occupies a GPU, including data waits and other overhead. *Service units (SU)* are the cluster's accounting charges. Queue time is reported separately and is not allocated GPU time. Active training time is measured per optimizer step with a

host-side monotonic clock (`time.perf_counter`), from the moment the data loader returns the batch to just after `torch.cuda.synchronize()` following the optimizer update. It therefore includes the host-to-device copy of the batch (pinned memory, non-blocking), setting the learning rate, the forward pass, the loss, the backward pass, the optimizer step and the host-side launch overhead of these operations. It excludes the wait for the next batch (logged as data-wait time), reading the loss value, logging, validation, checkpoint writes and start-up. After a crash the clock resumes from the last saved checkpoint, so lost steps are not counted. A milestone checkpoint is written after the first step at which the cumulative time reaches 1 h or 4 h (overshoot below one step).

Budget clock. Data are read from the cluster's shared file system, which delivers at most ∼98 training samples per second into one node and stalls for 5–22 minutes at a time, on all nodes at once. TFNO and AViT consume 116 and 330 samples per second of GPU time (Fig. 2), so a wall-clock budget would partly measure our storage. The 1-hour and 4-hour budgets count active GPU training time; waiting for data is logged separately.

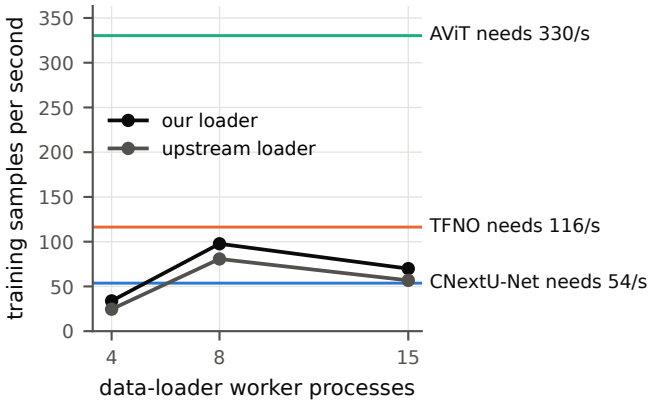


Figure 2: Training samples per second read from the shared file system by one node, against the rate at which each model consumes them on the GPU (horizontal lines).

Learning rate. An equal-budget search (20 minutes of GPU training per run, 10% data, validation only) over $\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}\}$ chose 10^{-3} for all three models (Table 3). It is the largest rate tried, so this identifies the best candidate tested, not an optimal learning rate. The initial grid is a bounded pilot search within the project's tuning budget: three models, three rates, and 20 minutes per trial require three hours of active GPU training in total. Rates above 10^{-3} were not evaluated, so their accuracy and stability are unknown. A larger learning rate may speed early learning, but it may also increase oscillation or destabilise training; no such failure is established here for an untested rate. The 20-minute ranking may also differ from the ranking after four hours. To preserve the current comparison, completed runs retain the selected rates. If additional tuning is performed, every family should receive the same additional search budget, using validation data only, and any changed configuration should be reported as a separate experiment.

Validation VRMSE after 20 min	10^{-4}	$5 \cdot 10^{-4}$	10^{-3}
CNextU-Net	0.841	0.469	**0.408**
TFNO	0.603	0.358	**0.302**
AViT	0.646	0.332	**0.257**

Table 3: Learning-rate search.

Batch size and precision. Batch 64 (The Well's) fits every model in fp32 on an A100 80 GB (peak 64.8 GB for CNextU-Net). The current TFNO implementation requires fp32 FFT operations, so all

models use fp32 for this comparison. This is an implementation constraint, not a claim that mixed-precision TFNO is impossible.

Pilot against actual. The pilots predicted 4 h of GPU training to take 6.0 / 8.0 / 9.7 h of wall time (CNextU-Net / TFNO / AViT) at the stall rate observed then; the 10% runs took 5.2–6.0 / 6.9–7.1 / 11.1–12.2 h.

4 Milestone 3: first 10%-data results

Training. The six runs completed their 4-hour budgets (Table 4, Fig. 3). On the fixed validation sample, AViT and CNextU-Net reach similar error after 4 hours; TFNO's loss plateaus after ∼1.5 hours. CNextU-Net's validation error peaks near one hour while its learning rate is still high; this temporal association does not establish the learning rate as the cause. The test evaluation below, on all test trajectories and both seeds, gives the rankings.

	Validation VRMSE (seed 0 / 1)		Epochs	Wall time	SU
Model	1 h	4 h	in 4 h	per run	per run
CNextU-Net	1.043 / 0.772	0.167 / 0.178	28	5.2–6.0 h	189–191
TFNO	0.508 / 0.524	0.310 / 0.325	61	6.9–7.1 h	274–286
AViT	0.468 / 0.418	0.161 / 0.167	175	11.1–12.2 h	443–488

Table 4: The 10% runs (140 training trajectories). Validation: 64 fixed windows, physical-field VRMSE.

The 64 validation windows are fixed and evenly spaced over the 34,300 one-step windows of the validation split (175 trajectories × 196), taken in the dataset's order (regime, trajectory, time): indices round(linspace(0, 34299, 64)). They cover all 35 regimes (one or two windows each) and 64 different trajectories; 18 of them (28%) are onset windows (first input frame < 50), close to the 26% of all windows. The learning-rate search used 512 windows chosen the same way (all 35 regimes, all 175 trajectories, 25% onset); the two samples share 8 windows. This monitoring sample does not replace the full test evaluation.

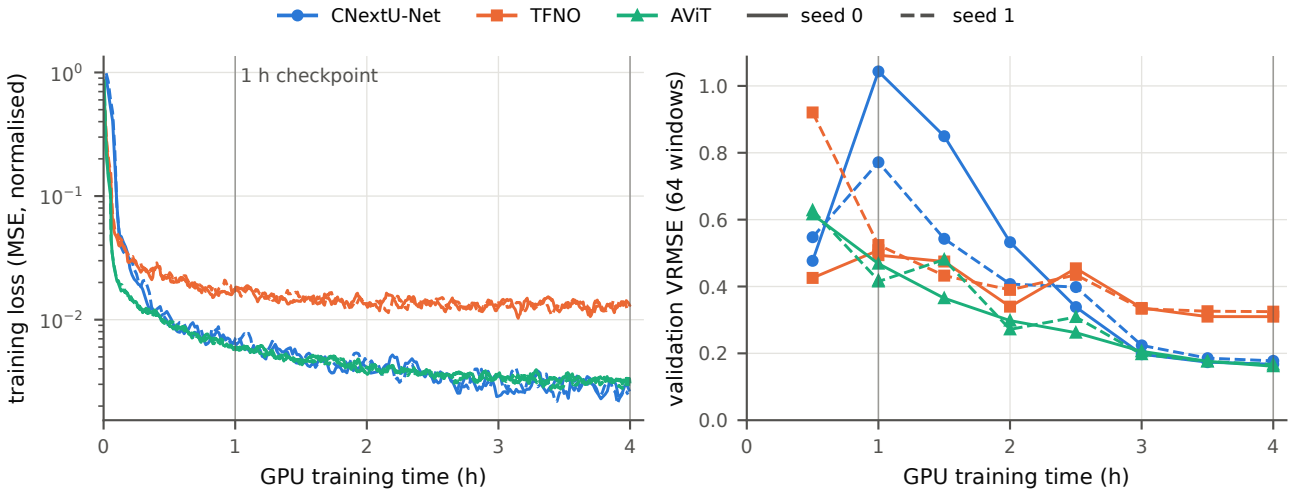


Figure 3: Training loss (left; smoothed over 3 minutes of training) and validation VRMSE on the 64 fixed windows (right) against active GPU training time, 10% data. Solid: seed 0; dashed: seed 1. Vertical lines: the 1-hour and 4-hour checkpoints.

Test evaluation (all 175 test trajectories, both seeds; Table 5, Fig. 4). After 4 hours AViT and CNextU-

Net have the same one-step error over all windows (0.134 and 0.136) and TFNO a higher one (0.209), but the two phases separate them: in developed convection CNextU-Net is clearly the most accurate (0.050, against 0.075 for AViT and 0.109 for TFNO), while AViT is the most accurate during onset. At 4 hours the two seeds agree to within 0.006; at 1 hour they do not (CNextU-Net: 0.85 and 0.60), and the order is reversed: AViT (0.39) and TFNO (0.47) are ahead of CNextU-Net (0.73), whose 1-hour checkpoint is far from converged. The released CNextU-Net, trained on all the data for longer, remains better in developed convection (0.038 against 0.050). Our TFNO, which uses its Fourier layers, is better than the released TFNO everywhere (developed 0.109 against 0.190; paper protocol 0.488 against 0.663) despite 10% of the data and 4 hours. Persistence is better than every model during onset and 2–4.5 times worse in developed convection.

	Physical-field one-step VRMSE			Paper protocol
Model (10% data, checkpoint)	all windows	onset	developed	(all windows)
CNextU-Net, 4 h	0.136 ± 0.006	0.387 ± 0.025	**0.050** ± 0.000	0.349 ± 0.022
TFNO, 4 h	0.209 ± 0.001	0.502 ± 0.003	0.109 ± 0.000	0.488 ± 0.001
AViT, 4 h	**0.134** ± 0.001	**0.308** ± 0.003	0.075 ± 0.000	**0.298** ± 0.002
CNextU-Net, 1 h	0.728 ± 0.176	2.618 ± 0.681	0.081 ± 0.003	2.345 ± 0.600
TFNO, 1 h	0.473 ± 0.078	1.505 ± 0.305	0.120 ± 0.000	1.362 ± 0.249
AViT, 1 h	0.394 ± 0.036	1.299 ± 0.140	0.085 ± 0.000	1.171 ± 0.125
Released CNextU-Net (100%, paper)	0.224	0.766	0.038	0.669
Released TFNO (100%, paper)	0.300	0.623	0.190	0.663
Persistence	0.199	0.121	0.226	0.202

Table 5: One-step test error: mean ± standard deviation over the two seeds (0.000: below 0.0005). Onset: first input frame < 50; developed: ≥ 50. Bold: best of our 4-hour models.

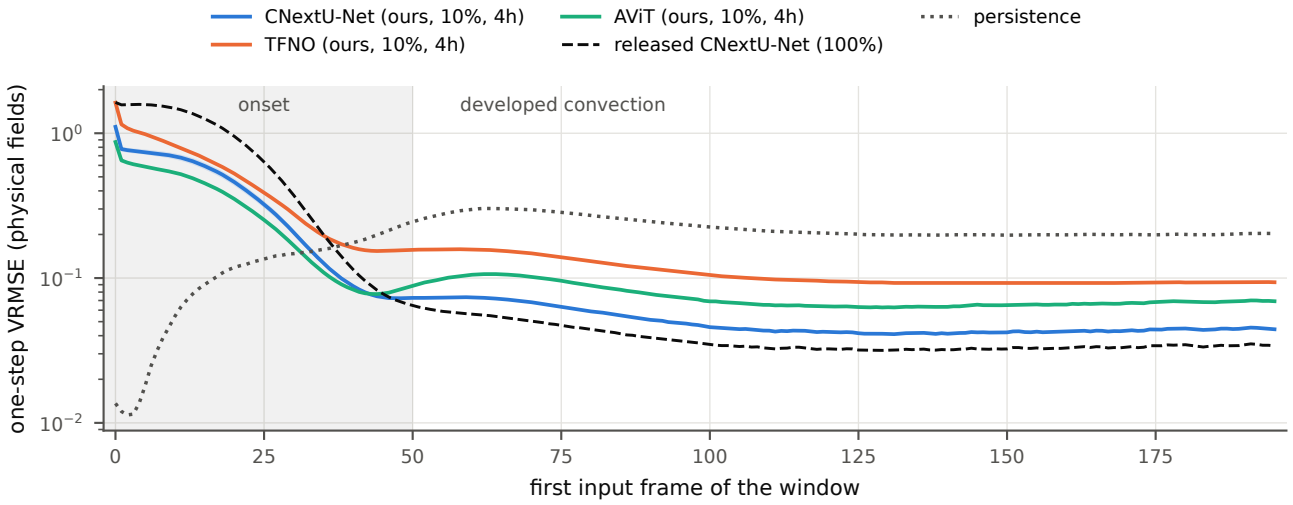


Figure 4: One-step test VRMSE (physical fields) by window, mean over the 175 test trajectories: our 10% models at 4 hours (seed mean; band: range over seeds), the released CNextU-Net trained on all the data, and persistence. Shaded: convection onset.

Rollouts (Table 6). One-step accuracy does not carry over to rollouts in the same order. CNextU-Net is the most stable: after 30 steps from frames 80 and 140 its error (0.53, 0.40) is below persistence's (0.86, 0.54), and 89% of its rollouts stay below VRMSE 1. AViT, despite matching CNextU-Net one step ahead, degrades fastest (0.80, 0.65; 63% below 1), and from frame 20, during onset, its error grows to 5.4. TFNO lies in between. Under the paper's frame-0 protocol all three land in the "> 10" range; CNextU-Net (15

/ 30) is below both released checkpoints.

Model	VRMSE after 30 steps from frame 20 / 80 / 140	Rollouts below VRMSE 1 / 2 / 5 after 30 steps	Paper protocol, steps 6–12 / 13–30 (normalised)
CNextU-Net, 4 h	1.78 / **0.53** / **0.40**	**0.89** / **0.94** / **0.98**	**15** / **30**
TFNO, 4 h	1.79 / 0.59 / 0.45	0.75 / 0.85 / 0.91	51 / 77
AViT, 4 h	5.42 / 0.80 / 0.65	0.63 / 0.80 / 0.87	75 / 167
Released CNextU-Net (100%)			36 / 35
Released TFNO (100%)			142 / 169
Persistence	0.68 / 0.86 / 0.54	0.94 / 1.00 / 1.00	0.27 / 0.50

Table 6: Rollouts on the test set, physical-field VRMSE unless stated, mean over the two seeds. Middle column: share of the 525 rollouts (175 trajectories × 3 start frames) whose error stays below each threshold for all 30 steps. Right: the paper's Table 3 protocol (published: > 10 in both windows).

Speed and memory (Table 7). In the same 4 hours of GPU training AViT takes six times as many optimizer steps as CNextU-Net and uses a seventh of the memory; TFNO is the fastest at inference.

Model	Optimizer steps in 4 h	Epochs	Peak training memory	Inference, per step	Per 30-step rollout
CNextU-Net	12,122–12,194	28	60.9 GB	8.3 ms	249 ms
TFNO	26,339–26,352	61	48.6 GB	4.4 ms	135 ms
AViT	74,978–75,060	175	8.8 GB	5.4 ms	163 ms

Table 7: Training throughput and memory (allocated, batch 64, fp32) and inference latency (batch 1), on one A100 80 GB.

Physical fidelity (4-hour checkpoints; Fig. 5). Each architecture errs at a different scale. As medians over the 35 regimes of the predicted-to-true energy ratio: CNextU-Net keeps the large and middle scales ($k_x < 100$) within 15% at every step and overshoots the small scales by a factor of 20–30. TFNO is too smooth: its middle-scale energy falls to 0.40 of the truth after one step and 0.12 after 30, while its small scales stay within a factor of 3; this is consistent with its spectral layers keeping only the 16 lowest Fourier modes in each direction ($k_x \lesssim 25$). AViT is accurate after one step at large and middle scales but accumulates energy during the rollout, reaching 5 times the truth at middle scales and over 700 times at small scales after 30 steps, in line with its fast degradation in rollouts. Persistence repeats its last input field, so its predicted spectrum is fixed within a rollout. Changes in its predicted-to-true energy ratio reflect the evolving true spectrum, not decay in the copied prediction. Ratios at the smallest scales should also be read alongside absolute spectra, because very small true energy can magnify the ratio.

What the predictions look like (Figs. 6 and 7): the true buoyancy field next to each model's rollout from the same four input frames, seed 0. At $\mathrm{Ra} = 10^8$ CNextU-Net follows the true plumes through 30 steps. TFNO blurs them and builds a spurious warm layer along the bottom wall. By step 12 AViT shows rectangular artifacts at the size of its 16×16-pixel patches, consistent with the energy it gains at middle and small scales (Fig. 5). At $\mathrm{Ra} = 10^{10}$ the true flow is far finer than any of the three reproduces; CNextU-Net keeps the large-scale plume pattern but smooths the small structures, and TFNO and AViT show the same failures as at 10^8.

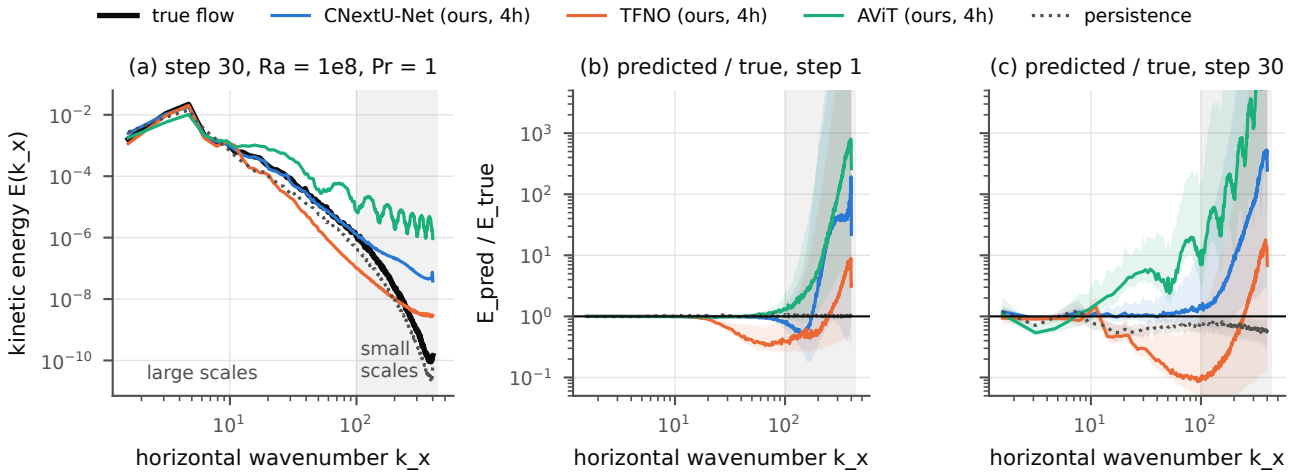


Figure 5: Horizontal kinetic-energy spectra. (a) True and predicted spectrum after 30 rollout steps for one regime ($\mathrm{Ra} = 10^8$, $\mathrm{Pr} = 1$). (b, c) Predicted over true spectrum after 1 and 30 steps: median over the 35 regimes (band: middle 50%); 1 is exact. Shaded: small scales ($k_x > 100$).

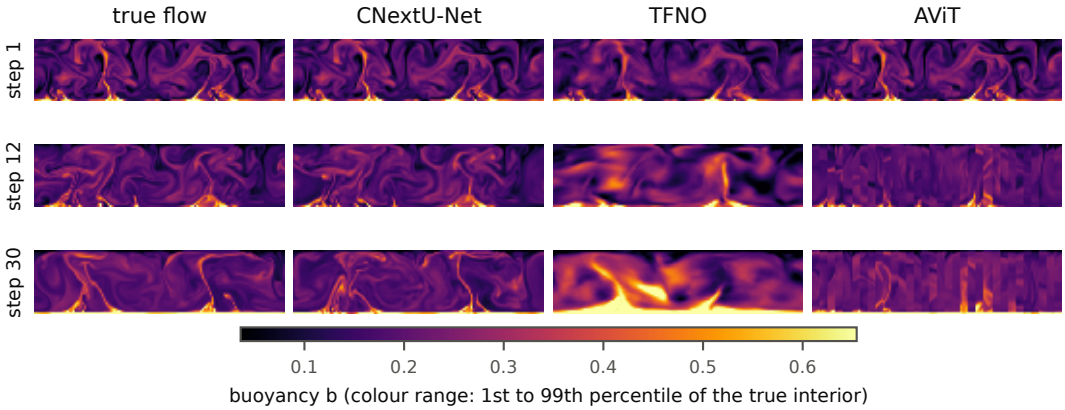


Figure 6: Buoyancy on a test trajectory at $\mathrm{Ra} = 10^8$, $\mathrm{Pr} = 1$, rolled out from frame 80: the true flow and our 10% / 4-hour models (seed 0) after 1, 12 and 30 steps. Bright: warm; dark: cold. The colour range covers the interior (1st to 99th percentile of the true field), so the hot bottom wall saturates.

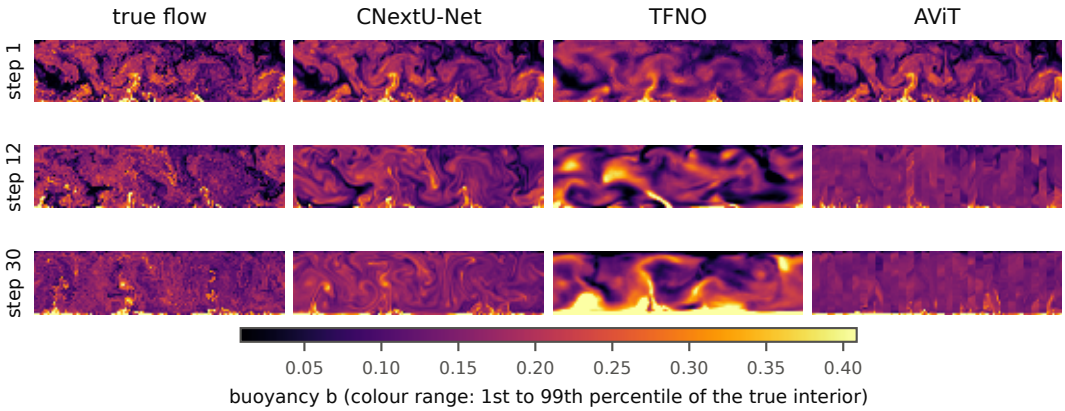


Figure 7: As Fig. 6, at $\mathrm{Ra} = 10^{10}$, $\mathrm{Pr} = 1$.

5 Revised plan

The *core comparison* remains three families, 10% and 100% training data, two seeds, and checkpoints at one and four hours. The main changes are:

- **Budget definition.** Experimental budgets count active GPU training time. Allocated GPU time, data waits, and SU are reported separately so that storage costs remain visible.
- **Evaluation.** Report the paper protocol for reference comparisons alongside the physical-field VRMSE (Section 1), separate onset from developed convection, and report the failure horizon at thresholds 1, 2 and 5 (defined in Section 1).
- **Core resource estimate.** The original 48 hours of active training are unchanged. The current working estimate is approximately 135 allocated GPU-hours for the core study and its supporting work, replacing the abstract's approximately 100-hour allowance. The 10% evaluations are now measured; the components are reconciled below. With all 10% work complete, Slurm accounting gives 72.6 allocated GPU-hours and 2,160 SU billed on the paid partition (class allocation), 2026-11-11. The six 10% runs contain 24 hours of active training and approximately 50 allocated GPU-hours.
- **Optional additions, outside the core estimate.** Consider two longer reference-recipe runs, one per CNextU-Net and TFNO, with the paper's learning rates and 24 h each of active GPU training (the same clock as the core runs; checkpoints at 6, 12, 18 and 24 h): 48 active GPU-hours, roughly 60–90 allocated given the data waits. They are not exact replications of the paper's 12-hour H100 experiments, and TFNO would run with the fixed forward pass, so they test the paper's recipe rather than the paper's exact model. A Dedalus [5] solver baseline at three resolutions is also optional and requires a feasibility check for restarting the solver from saved fields with the necessary state and boundary conditions. Its CPU cost is additional.

The estimate reconciles as follows (allocated GPU-hours from Slurm accounting on 2026-11-11, projections from Table 8):

Component	Allocated GPU-hours
Six 10% runs, completed (24.0 of them active training)	49.8
Learning-rate search, profiling, pipeline check, environment checks, first replication try	18.5
Test evaluation of the six 10% runs and persistence (completed)	4.2
Six 100% runs (projected, 24.0 active)	∼54–65
Test evaluation of the 100% runs	∼4.5
Core study	∼131–142

It contains no contingency, whereas the abstract's ∼100 hours included ∼30; a similar margin would bring it to ∼160–170. The optional additions above are excluded.

Table 8 gives the time per run: measured on the 10% runs and projected for the 100% runs. The validation set and the test evaluation do not depend on the data fraction. Time spent waiting in the queue (median 2.8 h, up to 15 h on the paid partition) is not included.

	10% runs, measured (hours)				Test eval.	100% runs
Model	training	data wait	other[a]	wall	(1 h + 4 h)	wall, projected
CNextU-Net	4.0	0.8–1.1	0.4–0.9	5.2–6.0	0.8–0.9 h	5.5–6.5 h
TFNO	4.0	2.5–2.8	0.3	6.9–7.1	0.5 h	7.5–9 h
AViT	4.0	6.1–7.3	0.9–1.0	11.1–12.2	0.3 h	14–17 h

Table 8: Time per run. [a]Startup, validation (64 windows every 30 minutes: 0.01–0.18 h per run) and checkpoint writes; for AViT also steps replayed after a file-system crash. Projection for 100%: the shared file system delivers at most ∼98 samples/s, below TFNO's and AViT's demand, and the full training split (∼290 GB) no longer fits in a node's page cache.

Remaining deliverables. The 10% evaluation is complete for all three models and both seeds, including failure horizons, spectra, latency and memory (Section 4). Remaining for the final report: the 100% runs and the data–compute comparison (10% against 100% at 1 and 4 hours), field-wise VRMSE and the Nusselt-number error per model, and the flow snapshots for the 100% runs.

Next steps, in priority order. (1) Improve data-loading robustness and assess node-local staging; (2) complete the six 100% runs (estimated approximately 2,400 SU) and the core data–compute comparison; (3) complete the promised metrics and the reproducible final report. Only after core completion, consider the abstract's extensions in their original order: input-noise robustness, a third seed, a 30% fraction, and ConvLSTM. The longer reference-recipe runs and solver baseline are further optional work and should not delay the core deliverables.

A Detailed training recipe

Sources of the reference settings: the paper (learning rates, 12-hour H100 budget) and The Well's training configuration at commit `e63dec8`:

- `configs/optimizer/adam.yaml`: AdamW, weight decay 10^{-4} (the paper's text says 0.01; we follow the code);
- `configs/data/rayleigh_benard.yaml`: batch 64;
- `configs/trainer/defaults.yaml`: 200 epochs;
- `configs/lr_scheduler/cosine_with_warmup.yaml` with `train.py`: 5 warmup epochs from 0.1·lr, then cosine to 0.1·lr.

Our software: The Well `e63dec8`, neuraloperator 0.3.0, PyTorch 2.8.0 (CUDA 12.8), Python 3.13; our repository at `f42b2e2` (training) and `4c0d3a8` (AViT resume, evaluation).

	This study	The Well paper / code
Activation	GELU (all three models)	GELU (same model code)
Optimizer	AdamW, $\beta = (0.9, 0.999)$, weight decay 10^{-4}, no clipping	AdamW, weight decay 10^{-4} in the code (0.01 in the text), no clipping
Loss	one-step MSE, normalised units, equal field weights	same
Batch size, precision	64, fp32	64, fp32
Learning rate	10^{-3} for all, from our equal-budget search	per model: $5 \cdot 10^{-4}$ (CNextU-Net), 10^{-4} (TFNO)
Schedule	500 warmup steps, cosine to 0 over the 4-hour budget	5 warmup epochs, cosine to 0.1·lr over 200 epochs, stopped at 12 h
Budget, hardware	1 h and 4 h of GPU training on one A100	12 h on one H100, best validation epoch
Training data	10% of the trajectories (100% next)	100%

Table 9: Training recipe: ours against the paper's.

References

[1] Ohana, R., et al. "The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning." NeurIPS, Datasets and Benchmarks Track, 2024.
https://arxiv.org/abs/2412.00568

[2] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. "A ConvNet for the 2020s." CVPR, 2022.
https://arxiv.org/abs/2201.03545

[3] Kossaifi, J., Kovachki, N., Azizzadenesheli, K., and Anandkumar, A. "Multi-Grid Tensorized Fourier Neural Operator for High-Resolution PDEs." TMLR, 2024.
https://arxiv.org/abs/2310.00120

[4] McCabe, M., et al. "Multiple Physics Pretraining for Spatiotemporal Surrogate Models." NeurIPS, 2024.
https://arxiv.org/abs/2310.02994

[5] Burns, K. J., Vasil, G. M., Oishi, J. S., Lecoanet, D., and Brown, B. P. "Dedalus: A flexible framework for numerical simulations with spectral methods." Physical Review Research 2, 023068, 2020.
https://arxiv.org/abs/1905.10388

[6] The Well source code, `https://github.com/PolymathicAI/the_well`: commits `2d8d7eaf` (2024-06-03) and `e4301602` (2025-11-17, "actually use the TFNO blocks"); issue #49, "Inconsistent Pre-trained Baseline Performance" (open since 2025-04-21).